

EasySave Technical Support Guide

Version 3.0 For ProSoft Support Staff Only

Overview

This document is intended for ProSoft support staff. It covers everything needed to diagnose and resolve issues with EasySave v3.0: system requirements, file locations, architecture and design patterns, new v3.0 subsystems (parallel execution, priority management, large-file semaphore, business software detection, CryptoSoft single-instance, Docker log centralization), log and state file formats, and the most common problems customers report.

EasySave is a .NET 10 application available as both a console app (EasySave.Console) and a WPF graphical interface (EasySave.WPF). Up to five backup jobs (full or differential) may be configured. Jobs now run in parallel. The EasyLog component is a separate DLL shared across ProSoft products.

Minimum System Requirements

Item	Detail
Operating System	Windows 10 or later (64-bit)
Runtime	.NET 10.0 must be installed separately if not already present
Disk Space	At least 50 MB for the application; additional space for log and state files
Permissions	Read access to source folders; Write access to target folders and %APPDATA%\EasySave\
Network	Required only when source or target paths point to a UNC share, or when log centralization is enabled
Docker (optional)	Docker Desktop (Windows) or Docker Engine (Linux/Server), required only for the log centralization feature
IDE (Dev)	Visual Studio 2022 or later
SDK (Dev)	.NET 10.0 SDK

File Locations

All runtime files are stored under the Windows user profile (%APPDATA%) so the application never needs administrator privileges.

File	Default Path	Notes
Executable (WPF)	EasySave.WPF.exe, customer-defined install path	Graphical interface entry point
Executable (Console)	EasySave.Console.exe, customer-defined install path	Console / CLI entry point
Job config	%APPDATA%\EasySave\jobs.json	Created automatically on first launch
Real-time state	%APPDATA%\EasySave\state.json	Overwritten live during every backup

File	Default Path	Notes
Daily log files	%APPDATA%\EasySave\Logs\YYYY-MM-DD.json (or .xml)	New file created each calendar day
Application settings	%APPDATA%\EasySave\appsettings.json	Includes all v3.0 settings
Language files	.\Resources\string-en.json, string-fr.json, string-es.json, string-ar.json	Must sit next to the executable
EasyLog DLL	EasyLog.dll, same folder as the executable	Loaded at runtime via standard .NET assembly resolution
CryptoSoft	%APPDATA%\EasySave\CryptoSoft\CryptoSoft.exe	Configurable via CryptoSoftPath in appsettings.json
Docker centralized logs	/data/logs/ (inside container, mapped to a named volume)	YYYY-MM-DD.json; single file per day across all machines

Note: %APPDATA% resolves to C:\Users\<username>\AppData\Roaming on a standard Windows installation.

Architecture & Design Patterns

Understanding the architecture is essential for diagnosing non-obvious bugs. EasySave v3.0 retains all patterns from v2.0 and adds several new concurrency and control-flow mechanisms.

MVVM (Model–View–ViewModel)

Layer	Class / File	Responsibility
Model	BackupJob, BackupManager, ConfigManager, StateManager, LocalizationManager, SettingsManager	All business logic, persistence, and domain state
ViewModel	MainViewModel (Console), MainViewModel (WPF)	Exposes commands and properties to the View; contains parsing and relay logic
View	ConsoleView (console), MainWindow.xaml (WPF)	Renders UI, reads user input, calls ViewModel commands only no business logic

Strategy Pattern (Backup Algorithms)

Backup types are implemented as interchangeable strategy objects behind IBackupStrategy:

```
interface IBackupStrategy { Task ExecuteBackupAsync(string source, string
target, BackupJob job); }
class FullBackupStrategy : IBackupStrategy // copies every file
unconditionally
class DifferentialBackupStrategy : IBackupStrategy // copies only
new/modified files
```

Both strategies implement the full v3.0 rule set: priority waiting, large-file semaphore, business software polling, and pause/cancel checking.

Observer Pattern (Real-Time State Updates)

BackupJob raises a standard .NET event after each file (throttled to 200 ms intervals):

```
public event EventHandler ProgressUpdated;
```

BackupManager subscribes StateManager.OnJobProgressUpdated before each job runs and unsubscribes in a finally block. If state.json is not updating, confirm that the subscription executes before job.Execute() is called.

Singleton Pattern (EasyLogger & LocalizationManager)

Singleton	Class	Access Point
Logger	EasyLogger (EasyLog.dll)	EasyLogger.Instance.WriteLog(entry)
Translations	LocalizationManager	LocalizationManager.Instance.GetString(key)

EasyLogger uses Lazy<T> for thread-safe lazy initialization and an internal _writeLock object to protect concurrent file writes from multiple parallel jobs. EasyLogger now also supports an HttpCentralizedLogWriter that POSTs entries to the Docker log server asynchronously without blocking the backup.

Repository / Interface Segregation (IConfigManager)

```
interface IConfigManager { void SaveJobs(List<BackupJob>); List<JobSaveData>
LoadJobs(); }
```

BackupManager accepts an optional IConfigManager in its constructor (defaults to ConfigManager if none provided). Unit tests inject a mock to avoid filesystem access.

v3.0 Subsystems

1. Parallel Backup

BackupManager.ExecuteAll() now launches all jobs concurrently using Task.WhenAll rather than a sequential loop. Each job runs its strategy on an async task from the thread pool.

```
List<Task> tasks = jobs.Select((_, i) => ExecuteJob(i + 1)).ToList();
await Task.WhenAll(tasks);
```

2. Priority File Management

Priority extensions are configured in AppSettings.PriorityExtensions. The mechanism uses a global atomic counter shared across all BackupJob instances:

Member	Type	Purpose
_globalPriorityFilesCount	static int (Interlocked)	Total priority files still pending across ALL running jobs
LocalPriorityFilesCount	int (per job)	Priority files still pending in THIS job
OthersHavePriority(myCount)	static bool	Returns true when global count > this job's local count, meaning another job has priority files pending
IncrementGlobalPriority()	static void	Called once per priority file discovered at job start
DecrementGlobalPriority()	static void	Called after each priority file finishes transferring

Non-priority files spin-wait (1-second polling) while BackupJob.OthersHavePriority() returns true. Priority files bypass the check entirely.

IMPORTANT: If state.json shows jobs stuck in Paused without user interaction, check whether priority files are deadlocked (e.g. an extension listed in PriorityExtensions that does not exist in the source folder, preventing the counter from reaching zero).

3. Large-File Semaphore

BackupJob.LargeFileSemaphore is a static SemaphoreSlim(1,1): exactly one large file may transfer at a time across all parallel jobs.

```
public static readonly SemaphoreSlim LargeFileSemaphore = new SemaphoreSlim(1, 1);
```

The threshold is AppSettings.MaxParallelSize (in KB, default 100). A file is considered "large" when its size in bytes exceeds MaxParallelSize * 1024. Small files never wait on the semaphore. The semaphore is released in a finally block to prevent deadlocks.

4. Real-Time Job Controls

Each BackupJob exposes three control methods callable from any thread:

Method	Mechanism	Effect
RequestPause()	ManualResetEventSlim (_pauseGate.Reset())	Strategy calls CheckPauseAndCancellation() between files; it blocks on _pauseGate.Wait(1000) until resumed.
RequestResume()	_pauseGate.Set()	Unblocks the waiting strategy thread; state returns to Active.
RequestCancel()	Interlocked.Exchange(_cancelRequested, 1) + _pauseGate.Set()	Strategy throws OperationCanceledException on next CheckPauseAndCancellation() call; state becomes Cancelled immediately.

Progress is throttled: NotifyProgress() suppresses events fired less than 200 ms apart (unless force: true is passed). This prevents UI/file I/O overload during rapid small-file sequences.

5. Business Software Detection

BusinessSoftwareService polls Windows process names at each file boundary. The list is stored in AppSettings.BusinessSoftwareName. The service strips .exe suffixes and performs case-insensitive matching via Process.GetProcessesByName().

Behaviour: if business software is detected before the job starts, the job logs a BLOCKED entry and returns immediately. If detected mid-job, the strategy enters a 1-second polling loop and sets the job state to Paused until the software closes, then restores Active and continues.

NOTE: The blocking software name is recorded in every log entry written while the job is blocked (SourceFilePath field contains "BLOCKED by: <name>", TransferTimeMs = -1).

6. CryptoSoft Single Instance

CryptoSoft.exe is now enforced as a single-instance application using a named Mutex. If a second instance attempts to start while one is already running, it exits immediately with a distinct error code. EasySave's EncryptionService detects this condition and surfaces it in the log as a negative encryptionTimeMs value.

EasySave manages CryptoSoft invocations sequentially within each job (one file at a time per job). Because multiple jobs may run in parallel, the EncryptionService relies on the single-instance guard to prevent simultaneous CryptoSoft invocations from different jobs on the same machine.

DIAGNOSTIC: If encryptionTimeMs is consistently -1 during parallel runs, confirm that the CryptoSoft mutex name matches between the CryptoSoft project and EncryptionService's launch logic. A name mismatch allows multiple instances and corrupts encrypted files.

7. Docker Log Centralization

The EasyLog DLL supports three LogDestinationMode values: Local, Centralized, and Both. The mode is set via EasyLogger.Instance.Configure(LogConfiguration).

Class	Location	Responsibility
LogDestinationMode	EasyLog/LogDestinationMode.cs	Enum: Local = 0, Centralized = 1, Both = 2
LogConfiguration	EasyLog/LogConfiguration.cs	Holds Destination, CentralizerBaseUrl, UseXmlFormat
EasyLogger	EasyLog/EasyLogger.cs	Routes WriteLog() calls to local writer, HttpCentralizedLogWriter, or both
HttpCentralizedLogWriter	EasyLog/HttpCentralizedLogWriter.cs	POSTs JSON payload to /api/logs endpoint; fire-and-forget; never throws to caller
Log Centralizer (Docker)	docker/log-centralizer/Program.cs	ASP.NET Core minimal API on port 8080 (exposed as 5080); appends to YYYY-MM-DD.json in /data/logs/

The Docker server appends each incoming log entry to the daily JSON file under a shared write lock. All machines write to the same file, identified by machineName and userName fields in every entry. A /health endpoint (GET) returns HTTP 200 to allow container health checks.

NOTE: HttpCentralizedLogWriter uses a fire-and-forget Task.Run(). Failed HTTP POSTs are silently discarded centralized logging must never interrupt a backup.

AppSettings Reference (appsettings.json)

Field	Type	Default	Description
Language	string	"EN"	Interface language code (EN, FR, ES, AR).
LogFormat	string	"JSON"	Daily log file format (JSON or XML).
LogDestination	string	"Local"	Log routing: Local, Centralized, or Both.
LogCentralizerUrl	string	"http://localhost:5080"	Base URL of the Docker log server.
ThemeMode	string	"Claire"	UI visual theme (Claire = Light, Sombre = Dark).
EncryptedExtensions	List<string>	[".txt", ".docx", ".pdf"]	File extensions passed to CryptoSoft.
CryptoSoftPath	string	Resources\CryptoSoft\CryptoSoft.exe	Path to CryptoSoft.exe. Stored as relative; resolved to absolute at runtime.
EncryptionKey	string	"default"	Plaintext encryption key. Change in production.
BusinessSoftwareName	List<string>	["CalculatorApp.exe"]	Process names that pause all jobs when detected.

Field	Type	Default	Description
PriorityExtensions	List<string>	[".jpg"]	Extensions treated as priority during parallel execution.
MaxParallelSize	long (KB)	100	Files exceeding this size (in KB) are serialised through LargeFileSemaphore.

Log File Formats

Daily Log Entry (JSON)

Each file transfer appends one entry to %APPDATA%\EasySave\Logs\YYYY-MM-DD.json (or the Docker server's equivalent). Example entry:

```
{
  "timestamp": "2025-05-17T14:22:01.123Z",
  "machineName": "WORKSTATION-01",
  "userName": "jdupont",
  "backupName": "Weekly Docs",
  "sourceFilePath": "C:\\Docs\\report.docx",
  "targetFilePath": "D:\\Backup\\report.docx",
  "fileSize": 204800,
  "transferTimeMs": 312,
  "encryptionTimeMs": 45
}
```

Field	Type	Notes
timestamp	ISO-8601 UTC	Set by EasyLogger if not provided by caller.
machineName	string	Environment.MachineName; populated automatically.
userName	string	Environment.UserName; populated automatically.
backupName	string	Job name as configured.
sourceFilePath	string	Full source path. Contains "BLOCKED by: <name>" for business-software pause events.
targetFilePath	string	Full target path. Empty for blocked/error entries.
fileSize	long (bytes)	0 for error entries.
transferTimeMs	long (ms)	Negative value indicates a failed transfer (exception during File.Copy).
encryptionTimeMs	long (ms)	0 = not encrypted; positive = CryptoSoft duration; -1 = CryptoSoft failed to start.

Real-Time State (state.json)

Field	Type	Notes
Name	string	Job name.
State	JobState enum	Inactive, Active, Paused, Completed, Cancelled, or Error.
TotalFiles	int	Total files discovered at job start.

Field	Type	Notes
FilesRemaining	int	Decrementated after each file (including skipped differentials).
TotalSize	long (bytes)	Total bytes discovered at job start.
SizeRemaining	long (bytes)	Decrementated per chunk during async copy.
CurrentSourceFile	string	Full path to the file currently being read.
CurrentTargetFile	string	Full path to where that file is being written.
LastUpdate	string	Formatted timestamp of the last NotifyProgress() call.

CryptoSoft Integration

Architecture

Class	Location	Responsibility
EncryptionService	EasySave.Core/Models/	Checks file extensions; launches CryptoSoft.exe as a child process; returns encryption duration via process exit code; detects single-instance conflicts.
SecurityHelper	EasySave.Core/Models/	Encrypts and decrypts the user's encryption key using Windows DPAPI (DataProtectionScope.CurrentUser).
SettingsManager	EasySave.Core/Models/	Loads and saves AppSettings (including CryptoSoftPath and EncryptionKey) to appsettings.json. Calls SecurityHelper.Unprotect on load, Protect on save.

Execution Flow

During a backup, the strategy calls `jobContext.Encryption.ShouldEncrypt(sourceFile)` for each file. If the extension matches, the file is first copied to the target location, then `jobContext.Encryption.Encrypt(targetFile, jobContext.EncryptionKey)` is called. CryptoSoft.exe receives the target path and key as quoted arguments and exits with elapsed milliseconds as its exit code.

v3.0 change: CryptoSoft now holds a named Mutex on startup. If a second instance tries to start (e.g. from a second parallel job on the same machine), it exits immediately. EncryptionService catches this case and returns -1, causing the backup to log a failure entry for that file.

Common Encryption Issues

Symptom	Diagnosis & Resolution
encryptionTimeMs is -1 in the log	CryptoSoft could not start, or a second instance was blocked by the single-instance mutex. Verify the executable exists at CryptoSoftPath and has execute permission. During parallel runs, check EncryptionService logic, it should serialize CryptoSoft calls within a job.
Files not encrypted despite matching extensions	Check that EncryptedExtensions contains the extension in lowercase with a leading dot (e.g. .pdf not pdf or .PDF). Confirm <code>jobContext.Encryption</code> is not null (injected by BackupManager.ExecuteJob).
EncryptionKey appears garbled in appsettings.json	Expected behaviour: the key is DPAPI-protected and stored as Base64. It can only be decrypted by the same Windows user account

Symptom	Diagnosis & Resolution
	on the same machine. If the user migrates to a new machine, the key must be re-entered.

Unit Tests

The EasySave.Tests project covers core logic. Run from the command line:

```
cd EasySave && dotnet test EasySave.Tests/EasySave.Tests.csproj
```

Test Class	What Is Covered
BackUpManagerTests	Job creation, 5-job limit, IConfiguration mock loading, ExecuteJob / ExecuteAll orchestration.
StrategyTests	FullBackupStrategy copies all files; DifferentialBackupStrategy skips unchanged files. Uses temporary directories.
LogTests	EasyLogger writes valid JSON; negative transferTimeMs preserved; concurrent writes do not corrupt output.
LogCentralizationTests	HttpCentralizedLogWriter payload format; local vs. centralized vs. both routing; Docker server endpoint contract.
SettingsTests	SettingsManager save/load round-trip; LogFormat, PriorityExtensions, MaxParallelSize, LogDestination persistence; DPAPI key protection verified.

Common Issues & Resolutions

Symptom	Diagnosis & Resolution
App starts then closes immediately	.NET 10.0 runtime not installed. Ask the customer to download it from microsoft.com/dotnet . Verify with: dotnet --version
Jobs disappear after restart	jobs.json missing or malformed. Check %APPDATA%\EasySave\. Delete the file to recreate a clean empty list (ConfigManager.LoadJobs catches parse exceptions and returns an empty list).
state.json not updating during backup	(a) Process lacks write access to %APPDATA%\EasySave\ check NTFS permissions. (b) Observer event was never subscribed confirm BackupManager.ExecuteJob() subscribes job.ProgressUpdated before calling job.Execute().
A job stuck in Paused state unexpectedly	Check whether PriorityExtensions contains extensions not present in the source folder (global counter never reaches zero). Also check BusinessSoftwareName for processes that are still running.
Log files not appearing	Check write access to %APPDATA%\EasySave\Logs\. The Logs subfolder is created by EasyLogger constructor on first run. EasyLogger._writeLock ensures concurrent writes are safe.
No files copied during backup	Full: source path empty or does not exist. Differential: all files already up to date (LastWriteTime on source <= target). UNC paths: confirm network connectivity.
transferTimeMs shows negative values	File.Copy() threw an exception. Inspect sourceFilePath in that log entry for permission issues, disk space problems, or locked files.
Interface in wrong language	Language preference is persisted in appsettings.json. Confirm the correct language code is saved. If language files are missing from

Symptom	Diagnosis & Resolution
	Resources\, LocalizationManager falls back to returning the key name in brackets (e.g. [MsgJobFinished]).
EasyLog.dll not found at startup	EasyLog.dll must be in the same directory as the executable. Verify: dir /b EasySave.WPF.exe EasyLog.dll
Docker log server not receiving entries	Confirm Docker is running (docker ps). Confirm LogCentralizerUrl in appsettings.json matches the server address and port. Check container logs: docker logs easysave-log-centralizer. POSTs are fire-and-forget; a 5-second timeout applies per request.
Centralized log file has duplicate date entries	The Docker server uses a file-level lock (writeLock object). Duplicate entries may indicate multiple Docker instances were started. Ensure only one container is running.

Localisation

All user-facing strings are externalised into JSON resource files loaded by the LocalizationManager singleton. If a translation key is missing, LocalizationManager.GetString() returns the key itself wrapped in square brackets (e.g. [MsgJobFinished]), making missing translations immediately visible without crashing.

File	Path	Encoding
string-en.json	.\Resources\string-en.json	UTF-8
string-fr.json	.\Resources\string-fr.json	UTF-8
string-es.json	.\Resources\string-es.json	UTF-8
string-ar.json	.\Resources\string-ar.json	UTF-8 (RTL)

Toolchain & Development Environment

Tool	Version	Purpose
.NET SDK	10.0	Compilation and unit tests
Visual Studio	2022 or later	Primary IDE
Docker Desktop / Engine	Latest stable	Log centralization container
docx library	9.x (npm)	Documentation generation